Smart Automatic Attendance Analyzer
System Specification

1.1 Project Overview
The Smart Automatic Attendance Analyzer System is a desktop utility designed to automate the aggregation and scoring of student attendance from disparate session logs. By processing directories of Microsoft Teams exports and Excel files, the system generates a unified performance report, providing instructors with tabular data and visual analytics.

1.2 Scope and Purpose
- Purpose: The primary objective is to eliminate manual data entry by programmatically calculating attendance scores based on student join times.
- Scope: Batch processing of directories containing .csv (UTF-16/UTF-8) and .xlsx files; dynamic threshold configuration; real-time search filtering; visual distribution analytics (Pie Charts); and CSV report generation.

1.3 Software Process Model
- Methodology: Agile / Incremental. Developed in short sprints to prioritize a functional MVC pipeline before adding visualization features.
- Version Control: GitHub was utilized for collaborative integration and source code management.
- Team Roles:
  - Backend: Watcharaphon (Bon) – Logic and Data Processing.
  - Frontend: Pongsathon (Ice) – UI/UX Design.
  - Controller/Integration: MAHER ABULKHAIR – System Wiring and Data Flow.

1.4 Technical Stack

- Language: Python 3.10+
- Core Libraries:
  - Pandas: Data manipulation and attendance aggregation.
  - Matplotlib: Generating visual analytics (Pie Charts).
  - Openpyxl: Support for modern .xlsx file formats.
  - Tkinter: Standard Python library for the Graphical User Interface.

1.5 Requirements Specification

- User Requirements (UR)
  - UR1: The user shall select a source directory to enable batch processing of all attendance files.
  - UR2: The user shall be able to filter the results in real-time using a search keyword.
  - UR3: The user shall be able to override default attendance thresholds (10m for Late, 30m for Absent).
- Functional Requirements (FR)
  - FR1: The system must parse .csv (MS Teams) and .xlsx formats.
  - FR2: The system must calculate scores using the formula:
  - Score=TotalSessions-(LateCount×0.5)-(AbsentCount×1.0)Score=TotalSessions-(LateCount×0.5)-(AbsentCount×1.0)..
  - FR3: The system must generate a Matplotlib Pie Chart showing the ratio of Normal, Late, and Absent sessions.
  - FR4: The system must export calculated results to a CSV file.

- Non-Functional Requirements (NFR)
  - Usability: The system must provide a Graphical User Interface (GUI).
  - Reliability: The system must handle empty directories or malformed files without crashing, providing clear error feedback via pop-up dialogs.

1.6 Environment & Dependencies

- Python Version: 3.10 or higher.
- Libraries: pandas, matplotlib, openpyxl, tkinter.

1.7 File Manifest

File Name	Role	Description
main.py	Entry Point	Initializes the MVC components and starts the application.
main_view.py	View	Handles UI layout, styling, and user-interaction events.
attendance_controller.py	Controller	Coordinates data flow between the UI and the Backend.
attendance_processor.py	Model (Logic)	Handles directory walking, file parsing, and scoring logic.
Participant.py	Model (Data)	Defines the Participant class and individual attendance counters.
report_generator.py	Utility	Handles the generation and formatting of CSV exports.

1.8 Execution Guide

- Install Dependencies: pip install pandas matplotlib openpyxl
- Run the Application: python main.py

Architectural Design

2.1 Architectural Pattern: Model-View-Controller (MVC)

The system is developed using the Model-View-Controller (MVC) architectural pattern. This design was chosen to enforce Separation of Concerns, ensuring that the business logic (scoring math) is independent of the user interface (GUI).

2.2 Component Breakdown

2.2.1 The Model (Business Logic & Data)

- Files: attendance_processor.py, Participant.py, report_generator.py
- Responsibility: This layer is the "brain" of the system. It owns the data structures and the mathematical formulas.
- Key Logic:
  - Parsing raw CSV/XLSX data into Participant objects.
  - Executing the scoring formula based on threshold configurations.
  - Aggregating class-wide statistics for analytics.
- Constraint: The Model has zero awareness of the GUI. It can be tested entirely in a terminal environment without launching a window.

2.2.2 The View (User Interface)

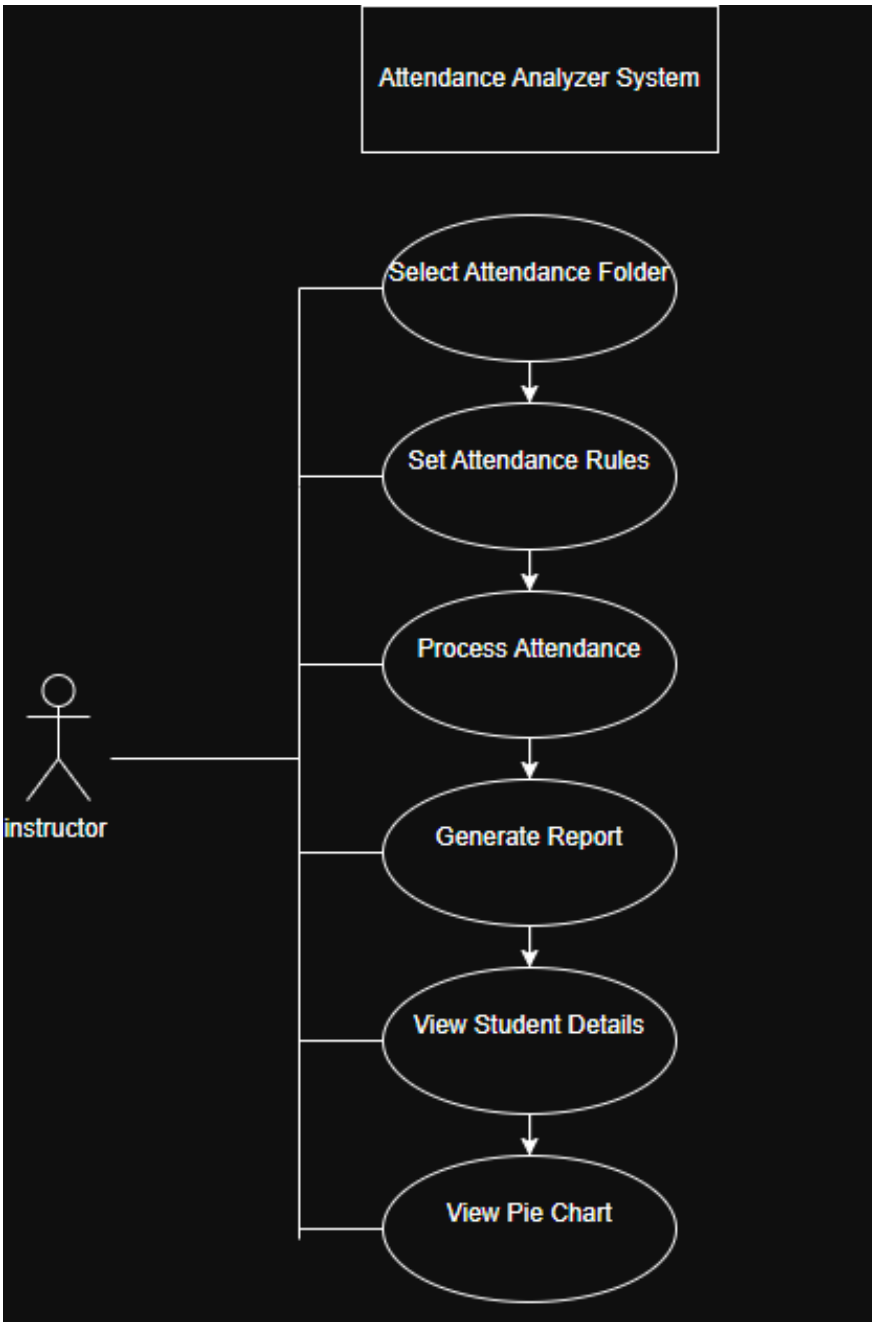
- File: main_view.py
- Responsibility: This layer is the "face" of the system. It is responsible for the layout, the dark-mode theme, and capturing user input (buttons, entries, search).
- Key Logic:
  - Rendering the ttk.Treeview table.
  - Providing a placeholder frame for Matplotlib charts.
  - Triggering events to the Controller when a user interacts with the UI.
- Constraint: The View does not perform any calculations. It is "passive" and only displays data provided to it by the Controller.

2.2.3 The Controller (Orchestration & Translation)

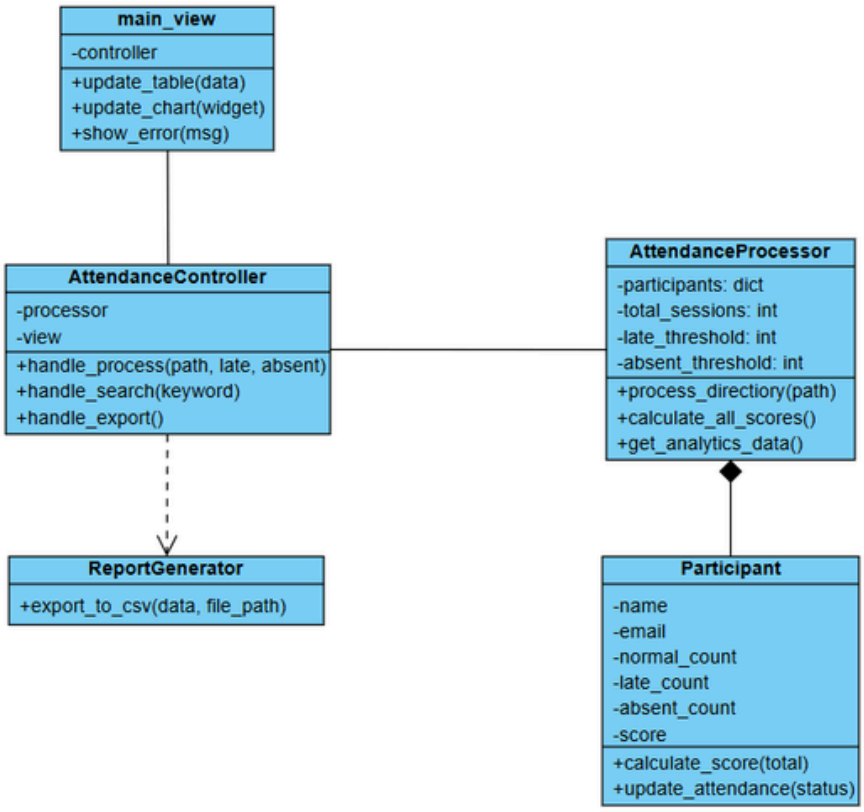
- File: attendance_controller.py
- Responsibility: This layer acts as the "glue" between the Model and the View. It manages the application’s state and coordinates the data flow.
- Key Logic:
  - Receiving the folder path and thresholds from the View.
  - Invoking the Model’s processing methods.
  - Data Transformation: Translating complex Backend objects into simplified dictionaries that the View can easily display in the table.
  - Handling the "Save" dialogs for CSV exports.

System Modeling

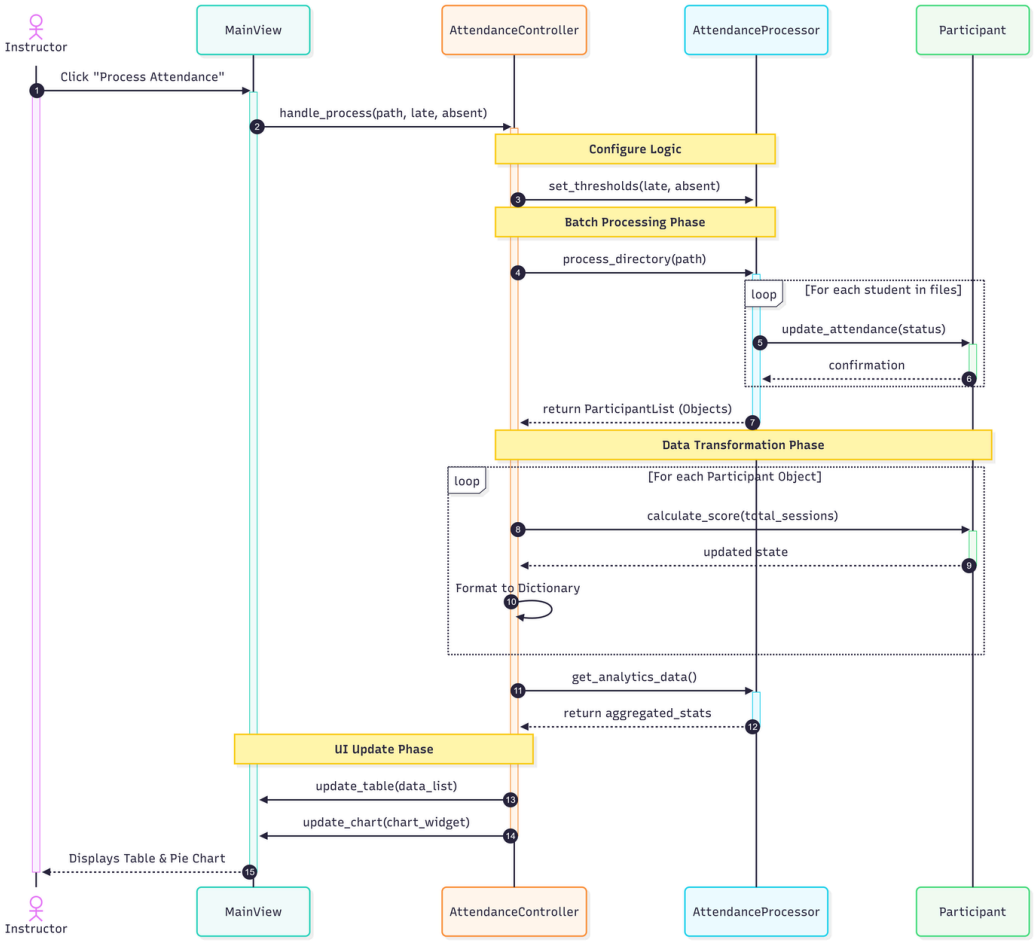
Use Case Diagram:



Class Diagram:



Sequence Diagram:



Quality Assurance & Testing

4.1 Testing Strategy

The system underwent Black Box Testing to ensure that all functional requirements were met. We focused on data accuracy, edge-case handling (empty folders), and UI responsiveness.

4.2 Test Case Matrix

ID	Test scenario	Input	Expected Resuit	Status
TC1	Happy Path	Select folder with 5 valid MS Teams CSVs.	Table populates with 100% accurate scores; Pie chart displays.	Pass
TC2	Threshold Logic	Set Late to 5m. Process student who joined at 6m.	Student status marked as "Late"; Score deducted by 0.5.	Pass
TC3	Search Filter	Enter "Watcharaphon" in search bar.	Table instantly filters to show only matching rows.	Pass
TC4	Edge Case: Empty	Select a folder with no .csv or .xlsx files.	System shows error popup; No crash occurs.	Pass
TC5	CSV Export	Click "Export" and save as results.csv.	CSV file is created; Data matches the GUI table exactly.	Pass

Conclusion: Lessons Learned

5.1 Technical Reflection

The development of the Smart Attendance Analyzer highlighted the importance of a clean architecture. By utilizing the MVC Pattern, the team was able to:

- Successfully manage a major UI refactor without affecting the core mathematical logic.
- Handle complex data formatting issues (UTF-16 encoding from MS Teams) within the Model layer, keeping the Controller and View simplified.

5.2 Agile Adaptability

Following the Agile/Incremental methodology allowed the team to pivot during Day 7. When time constraints made the PDF export high-risk, we utilized our sprint review to prioritize a high-stability CSV export over a potentially buggy PDF feature. This ensured a "Feature Complete" product for the final deadline.

5.3 Final Summary

The system meets all primary user requirements. It provides a reliable, user-friendly solution for instructors to analyze attendance data, significantly reducing the administrative overhead of manual score calculation.